

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 7 Pt 2.

Procesador CISC 68HC11

Fecha de Entrega

20 de Noviembre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Obtener el mayor de cinco números de 8 bits sin signo y guardarlo en `Res`, mediante un programa para el procesador CISC 68HC11 y la implementación del salto condicional `JAE`.

2. Desarrollo

Se agregó `JAE` en `Secuenciador/memory.vhd` y se cargaron el programa y los datos en RAM mediante `mem_content.mif`. Se reutilizaron las instrucciones de carga, comparación y almacenamiento del proyecto de Quartus.

2.1. Lógica para obtener el número mayor

A conserva el mayor encontrado y B recibe el siguiente elemento. Se carga $L[0]$ en A y se compara con los otros cuatro valores: si $B > A$, se actualiza A; al terminar, se copia a `Res`.

Código 1: Lógica del algoritmo para una lista de cinco números.

```
L = [0x05, 0x09, 0x01, 0x02, 0x06];
mayor = L[0];
for (i = 1; i < 5; i++) {
    if (L[i] > mayor) {
        mayor = L[i];
    }
}
Res = mayor;
```

En ensamblador, el ciclo del Código 1 se implementó con cuatro bloques consecutivos, sin contador de iteraciones. Los datos se representan en hexadecimal y se interpretan sin signo, de 0 a 255.

2.2. Instrucciones utilizadas

La Tabla 1 resume las instrucciones del programa. La única rutina nueva fue `JAE`, con el opcode `7C`.

Tabla 1: Instrucciones empleadas para encontrar y almacenar el mayor.

Opcode	Instrucción	Modo	Función en el programa
`96`	`LDAA`	EXT	Cargar el primer elemento en A o actualizar el mayor.
`F6`	`LDAB`	EXT	Cargar el siguiente elemento de la lista en B.
`1C`	`CMP`	INH	Calcular $A - B$ para actualizar las banderas, sin modificar los acumuladores.
`7C`	`JAE`	EXT	Saltar si $A \geq B$, omitiendo la actualización de A.
`A7`	`STAA`	EXT	Guardar el mayor de A en `Res`.
`7E`	`JMP`	EXT	Mantener el programa en un ciclo final.

Cada bloque ejecuta `LDAB`, `CMP` y `JAE`. Si $A \geq B$, el salto omite el `LDAA` siguiente; si $A < B$, esa carga actualiza el mayor.

2.3. Comparación y bandera de acarreo

`CMP` calcula $A - B$ y actualiza las banderas sin modificar los acumuladores. En esta UPA, el bit adicional de la resta determina C:

$$C = \begin{cases} 1, & A < B, \\ 0, & A \geq B. \end{cases}$$

Por ello, `JAE` toma el salto cuando $C = 0$. Se usa C porque la comparación es sin signo; N por sí sola no determina el orden en todo el intervalo de 8 bits. Las señales con prefijo `n` se habilitan en bajo; las demás señales indicadas en la Tabla 2 valen 1.

Tabla 2: Señales de control utilizadas por `CMP`.

Estado	Señales	Función
`1C0`	EA1, EA0, EB1, EB0, UPA5, UPA0, selmux, ACCSEC	EA y EB conectan los acumuladores a la UPA. UPA0 selecciona A y B; UPA5 selecciona la resta. selmux y ACCSEC fijan $C_{in} = 1$ para obtener $A - B$.
`1C1`	nOEUPA=0, nDUPA=0, CC, CN, CV, CZ	Se conserva la salida registrada de la UPA y se cargan las banderas. nWA=nWB=1 evitan modificar los acumuladores.
`1C2`	nEPC2=0, nCBD=0	Se prepara la siguiente búsqueda y se regresa a `009`.

2.4. Implementación de JAE

Se copiaron las seis microinstrucciones de `JE` a `7C0`–`7C5`, conservando las transferencias y cambiando la condición y la liga en `7C3`. MAP dirige el opcode `7C` a `7C0`; después se leen los dos bytes del destino, primero el alto y luego el bajo, en RA.

La palabra de microprograma tiene 98 bits: 20 de secuenciamiento y 78 de control. En `7C3` se definieron estos campos:

$$\underbrace{10000}_{\text{prueba de } C}\ \underbrace{1}_{VF}\ \underbrace{01}_{\text{salto condicional}}\ \underbrace{0111\,1100\,0101}_{\text{liga } \texttt{7C5}}.$$

El secuenciador evalúa $CC_{\text{sec}} = \overline{C \oplus VF}$. Con instrucción `01`, carga la liga si hay coincidencia; de lo contrario, continúa al siguiente microestado:

- $C = 1$ ($A < B$): salta a `7C5`, omite la carga del PC y continúa con el `LDAA` que actualiza A.
- $C = 0$ ($A \geq B$): ejecuta `7C4`, carga PC $\leftarrow$ RA y omite la actualización de A.

Así, saltar internamente a `7C5` evita tomar el salto del programa. CC_{sec} es la condición del secuenciador; la señal `CC` de `CMP` habilita la carga de la bandera de acarreo.

Tabla 3: Transferencias y señales de control de `JAE` (EXT).

Estado	Transferencia	Señales y motivo
`7C0`	Reg. Dir ← PC	nEPC2=0 habilita el PC en el bus de direcciones; nCBD=0 captura su valor.
`7C1`	RA_{ALTA} ← Mem; PC ← PC+1	nAS=0 y nRW=1 leen memoria. BD=1, nERA1=0 y RA2=1 cargan el byte alto. PC0=1 incrementa el PC.
`7C2`	Reg. Dir ← PC	nEPC2=0 y nCBD=0 preparan la dirección del segundo byte.
`7C3`	RA_{BAJA} ← Mem; PC ← PC+1; prueba de C	nAS=0 y nRW=1 leen memoria. BD=0, nERA0=0 y RA1=RA0=1 cargan el byte bajo. PC0=1 incrementa el PC.
`7C4`	PC ← RA, sólo si $C = 0$	nERA1=nERA0=0 habilitan RA en ambos buses. nEPC1=nEPC0=0 y PC2=PC0=1 cargan los 16 bits del PC.
`7C5`	Reg. Dir ← PC	nEPC2=0 y nCBD=0 preparan la búsqueda de la siguiente instrucción.

`7C4` conserva la prueba de interrupciones `01111/1/11/000`; `7C5` usa `11000/0/01/009` para volver a la búsqueda. El Código 2 muestra las palabras agregadas antes del `else` final de `memory.vhd`.

Código 2: Microprograma agregado para JAE con opcode 7C.

```
-- JAE (EXT): salta si A >= B, es decir, C = 0
elsif(dir= X"7C0") then data <=
  "00000" & "0" & "00" & "000000000000" &
  "10010010000000000011011100001110001110000110000111111000000000000000000000000010";
elsif(dir= X"7C1") then data <=
  "00000" & "0" & "00" & "000000000000" &
  "10010010000000000011011100001011001110001110011111101100000000000000000000000010";
elsif(dir= X"7C2") then data <=
  "00000" & "0" & "00" & "000000000000" &
  "10010010000000000011011100001110001110000110000111111000000000000000000000000010";
elsif(dir= X"7C3") then data <=
  "10000" & "1" & "01" & "011111000101" &
  "10010010000000000011011100001100111110001110011111101000000000000000000000000010";
elsif(dir= X"7C4") then data <=
  "01111" & "1" & "11" & "000000000000" &
  "10010010000000000011011100001000001110001001011111111000000000000000000000000010";
elsif(dir= X"7C5") then data <=
  "11000" & "0" & "01" & "000000001001" &
  "10010010000000000011011100001110001110000110000111111000000000000000000000000010";
```

2.5. Programa y distribución de la RAM

El programa ocupa las posiciones decimales 0–48, la lista las posiciones 49–53 (`0031`–`0035`) y `Res` la posición 54 (`0036`). Cada `JAE` apunta al siguiente bloque o al almacenamiento final. Las direcciones del Código 3 están en hexadecimal.

Código 3: Programa en ensamblador para obtener el mayor de cinco elementos.

```
Dir.   Bytes      Instruccion
0000   96 00 31   LDAA $0031    ; A = L[0]
0003   F6 00 32   LDAB $0032    ; B = L[1]
0006   1C         CMP           ; comparar A-B
0007   7C 00 0D   JAE  $000D    ; conservar A si A >= B
000A   96 00 32   LDAA $0032    ; actualizar el mayor
000D   F6 00 33   LDAB $0033    ; B = L[2]
0010   1C         CMP
0011   7C 00 17   JAE  $0017
0014   96 00 33   LDAA $0033
0017   F6 00 34   LDAB $0034    ; B = L[3]
001A   1C         CMP
001B   7C 00 21   JAE  $0021
001E   96 00 34   LDAA $0034
0021   F6 00 35   LDAB $0035    ; B = L[4]
0024   1C         CMP
0025   7C 00 2B   JAE  $002B
0028   96 00 35   LDAA $0035
002B   A7 00 36   STAA $0036    ; Res = A
002E   7E 00 2E   JMP  $002E    ; ciclo final
```

`STAA $0036` sobrescribe `Res` con A: en `A74`, nERA2=0 y nCBD=0 cargan la dirección desde RA; en `A75`, EA0=1 coloca el dato en el bus y nAS=nRW=0 permiten escribirlo en RAM. Después, `JMP $002E` repite el salto sin modificar el resultado y evita ejecutar la lista como instrucciones.

En `mem_content.mif` se usan `WIDTH=8`, `DEPTH=256`, `ADDRESS_RADIX=UNS` y `DATA_RADIX=HEX`. Las posiciones 0–48 contienen los bytes del Código 3; el fragmento siguiente completa los datos y el relleno de la RAM:

Código 4: Datos y relleno final del archivo MIF para el primer ejemplo.

```
    49 : 05; -- L[0]
    50 : 09; -- L[1]
    51 : 01; -- L[2]
    52 : 02; -- L[3]
    53 : 06; -- L[4]
    54 : 00; -- Res: valor inicial
    [55..255] : 00;
END;
```

3. Simulación

Se simuló en Quartus mediante ModelSim con un reloj de 10 ns, `RESET=1` e `IRQn=XIRQn=1`. Se observaron los acumuladores, RA, PC, `Edo_Pres` y `csnn`. Se amplió la duración para incluir la última comparación, la escritura y el ciclo final; las transferencias se reflejan un ciclo después del estado que genera sus señales.

3.1. Ejemplo 1: lista [05,09,01,02,06]

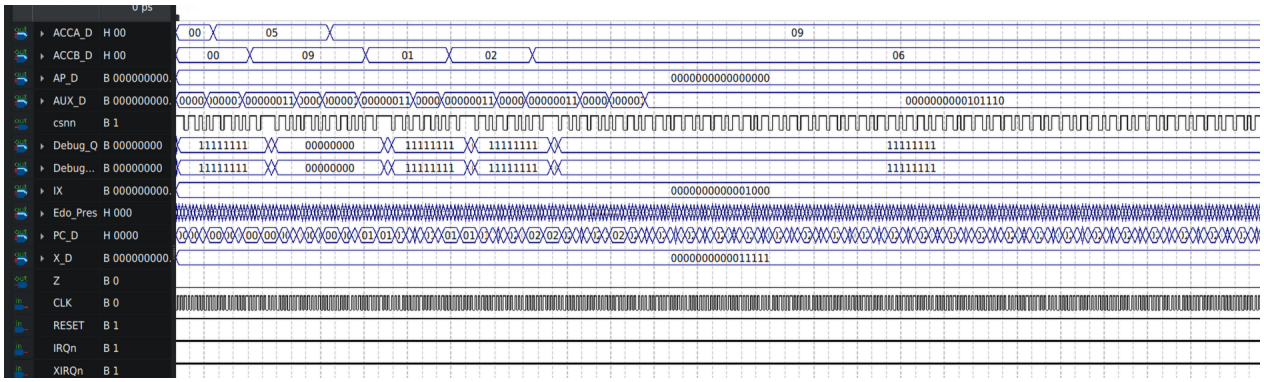


Figura 1: Simulación de la búsqueda del mayor para la lista [05,09,01,02,06].

En la Figura 1, A carga `05` y cambia a `09` al compararse con el segundo elemento. Después conserva `09` frente a `01`, `02` y `06`, como resume la Tabla 5. Finalmente, `STAA` guarda `09` en `Res`.

Tabla 4: Comparaciones realizadas en el primer ejemplo.

A antes	B	C	Acción	A después
05	09	1	Actualizar A.	09
09	01	0	Conservar A.	09
09	02	0	Conservar A.	09
09	06	0	Conservar A.	09

3.2. Ejemplo 2: lista [03,02,09,0C,0B]

Se cambiaron únicamente los cinco datos de la lista, conservando el programa y las direcciones.

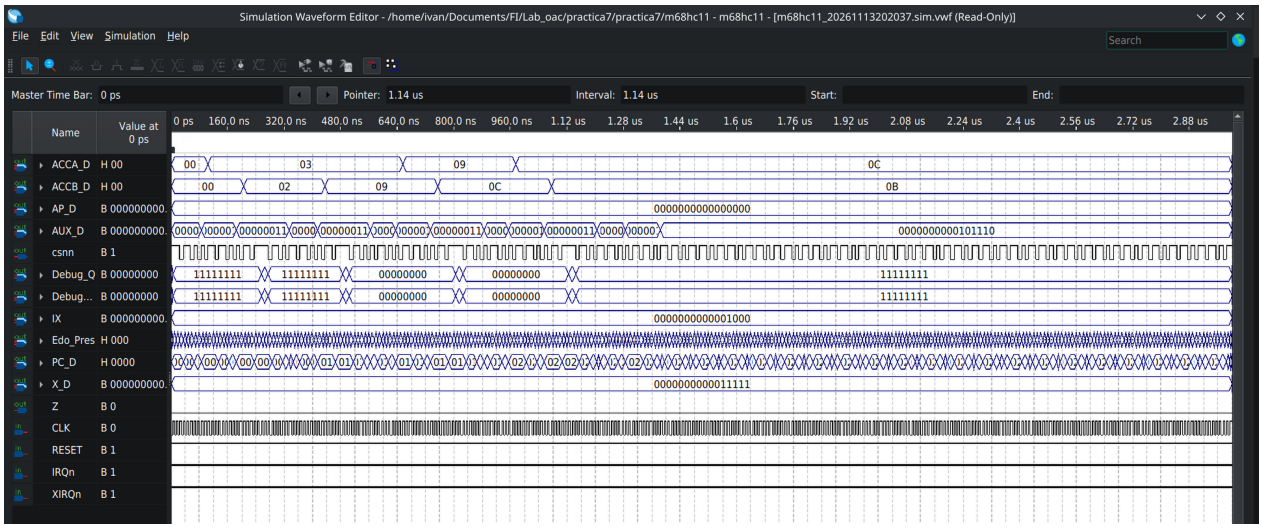


Figura 2: Simulación de la búsqueda del mayor para la lista [03,02,09,0C,0B].

En la Figura 2, A conserva `03` frente a `02`; luego se actualiza a `09` y a `0C`. La última comparación con `0B` conserva `0C`, que se guarda en `Res`. La Tabla 6 muestra las cuatro decisiones.

Tabla 5: Comparaciones realizadas en el segundo ejemplo.

A antes	B	C	Acción	A después
`03`	`02`	0	Conservar A.	`03`
`03`	`09`	1	Actualizar A.	`09`
`09`	`0C`	1	Actualizar A.	`0C`
`0C`	`0B`	0	Conservar A.	`0C`

3.3. Verificación del almacenamiento

Las capturas muestran el mayor en A; `Res` es el contenido de RAM en `0036`. Su escritura se verifica en `A75`, con nAS=nRW=0, dirección `0036` y el dato de A. `csnn` por sí sola no distingue lectura de escritura.

Después, el `JMP` final carga RA con `002E`, visible como `0000000000101110` en `AUX_D`. El PC repite `002F`, `0030`, `0031` y `002E`; `0031` es transitorio y la lista no se ejecuta. El resultado permanece en RAM: `09` para la primera prueba y `0C` para la segunda.

4. Conclusión

Percival Ulises Espinoza Matamoros. En esta práctica pude relacionar la búsqueda del mayor de una lista con las instrucciones que ejecuta el procesador. La implementación de `JAE` me permitió comprender cómo las banderas y el secuenciador deciden si se conserva o se actualiza un dato. Al probar dos listas, observé que el programa obtiene distintos resultados sin cambiar sus instrucciones.

Oscar Iván Montiel Juárez. En esta práctica entendimos de manera más clara cómo una comparación puede cambiar el recorrido de nuestro programa. Al implementar `JAE` relacionamos la bandera de acarreo con las líneas de control que permiten cargar el PC. He de mencionar también que revisar los cambios de A y B en la simulación nos ayudó a comprobar cómo se conserva el mayor y cómo se guarda finalmente en `Res`.

Amy Valentina Veraza García. Esta práctica me permitió comprender la importancia de elegir la bandera adecuada para comparar números sin signo. Con `CMP` y `JAE` fue posible conservar el mayor de cinco elementos y almacenarlo en RAM. Las simulaciones ayudaron a distinguir entre el valor calculado en el acumulador, su almacenamiento en memoria y el ciclo que mantiene el resultado al finalizar.